Reinforcement Learning for Autonomous Lunar Landing

Reward Shaping, Policy Optimization, and Reward Hacking

Raja Halazon

Mechanical Engineering, McGill University

October 2026

ENVIRONMENT	ALGORITHM	EVIDENCE BASE
Gymnasium LunarLander-v3	PPO, Stable-Baselines3	4 agents, 400 evaluation episodes

Report version 2.0. All quantitative results were recomputed from the original evaluation dataset (final_results.csv). This is a simulation study; no flight hardware or flight software is described.

Abstract

This report examines how the specification of a reward function shapes the behavior of a learned controller for a simulated lunar-landing task. Four agents were trained with Proximal Policy Optimization (PPO) in the Gymnasium LunarLander-v3 environment, one training run per reward condition and approximately 5×10^5 environment steps per run: the unmodified environment reward; the same reward with a -0.01 per-step time penalty; a deliberately flawed $+1$ per-step bonus for a slow, upright state; and a corrected term that rewards a change in a stability measure rather than its continued satisfaction. Each trained policy was evaluated with deterministic actions on the same 100 environment seeds, using the unmodified reward. The baseline achieved a mean score of 177.5 points (95% confidence interval 156.9 to 197.0) and came to rest in 75 of 100 episodes. The time-penalty agent landed in 27 episodes. The hover-bonus agent landed in none, reached the 1,000-step time limit in 87 episodes, and spent on average 86.6% of each episode in the slow, upright, airborne state, a pattern consistent with reward hacking. The corrected agent landed in 55 episodes (mean score 90.2): 55 percentage points more than the hover-bonus agent on the same seeds, but 20 points fewer than the baseline. Because each reward condition was trained once, the reported intervals describe variability across starting conditions for four specific trained policies, not the expected effect of each reward design; the time-penalty result in particular cannot be separated from training variability. Within these limits, the study shows that a per-step bonus for a proxy state can displace the task objective, and it specifies the additional evidence needed to establish the reward effects causally.

Keywords: reinforcement learning; proximal policy optimization; reward shaping; reward hacking; specification gaming; autonomous landing; evaluation methodology.

Contents

Abstract ii

1 Introduction 1

2 Project Objectives and Research Questions 1

3 Theoretical Background: Reinforcement Learning 2
- 3.1 Markov decision processes . . . 2
- 3.2 Termination and truncation . . . 3
- 3.3 Reward shaping . . . 3
- 3.4 Specification gaming and reward hacking . . . 4

4 Proximal Policy Optimization and Neural Network Architecture 4
- 4.1 Clipped policy optimization . . . 4
- 4.2 Implementation and hyperparameters . . . 4
- 4.3 Policy and value networks . . . 5

5 LunarLander Simulation Environment 5

6 State Space, Action Space, and Reward Mechanism 6
- 6.1 Observation and action spaces . . . 6
- 6.2 Base reward . . . 7

7 Experimental Setup and Training Methodology 7
- 7.1 Software and hardware . . . 7
- 7.2 Training procedure . . . 7
- 7.3 Provenance of the evidence . . . 8

8 Design of the Four Reward Functions 8
- 8.1 Agent 1: baseline . . . 9
- 8.2 Agent 2: time penalty . . . 9
- 8.3 Agent 3: hover reward . . . 9
- 8.4 Agent 4: corrected reward . . . 9

9 Evaluation Methodology and Performance Metrics 10
- 9.1 Protocol . . . 10
- 9.2 Metrics . . . 10
- 9.3 Statistical methods . . . 11

10 Experimental Results 11
- 10.1 Task performance . . . 11
- 10.2 Landing outcomes . . . 12

11 Statistical and Graphical Analysis 13
- 11.1 Distribution of scores . . . 13
- 11.2 Episode duration . . . 14
- 11.3 Slow, upright, airborne behavior . . . 14
- 11.4 Duration and score . . . 16
- 11.5 Paired comparisons . . . 16
- 11.6 Uncertainty . . . 17

12 Reward Hacking: Mechanism and Observed Behavior 17
- 12.1 Mechanism (analysis) . . . 17
- 12.2 Observed behavior (verified) . . . 18
- 12.3 Interpretation and its limits . . . 19

13 Corrected Reward Design and Performance Recovery **20**
13.1 Recovery relative to the hover reward . . . 20
13.2 Comparison with the baseline . . . 20
13.3 Residual behavior . . . 21
13.4 Why the correction fell short of the baseline . . . 21
14 Engineering Discussion and Interpretation **21**
14.1 Strength of the evidence . . . 21
14.2 The time penalty . . . 21
14.3 Lessons for reward specification . . . 22
15 Experimental Limitations and Reproducibility **23**
15.1 Limitations . . . 23
15.2 Reproducibility status . . . 23
16 Recommendations for Future Work **24**
17 Conclusions **24**
References **25**
Technical Appendices **27**
Appendix A. Dataset Description and Integrity Checks . . . 27
Appendix B. Derivations . . . 27
Appendix C. Observation Scaling and Threshold Conversion . . . 28
Appendix D. Reproduction of the Analysis . . . 28
Appendix E. Cross-Check of the Preliminary Summary . . . 28

List of Figures

1 Agent–environment interaction (conceptual) . . . 3
2 PPO training and evaluation workflow (conceptual) . . . 6
3 Mean evaluation score by agent . . . 12
4 Landing rate and episode outcomes . . . 13
5 Distribution of episode scores . . . 14
6 Episode duration by agent . . . 15
7 Slow/upright airborne behavior . . . 15
8 Episode duration versus score . . . 16
9 Reward-hacking mechanism (conceptual) . . . 18
10 Accumulation of the added training reward (analytical) . . . 19
11 Seed-paired scores: baseline versus corrected reward . . . 20

List of Tables

1 Research questions and evidence . . . 2
2 Assumed PPO hyperparameters . . . 5
3 Observation vector of LunarLander-v3 . . . 7
4 Software environment . . . 8
5 The four reward designs . . . 8
6 Evaluation scores . . . 12
7 Episode outcomes and durations . . . 12
8 Score and duration conditioned on landing . . . 13
9 Slow/upright airborne frames per episode . . . 15
10 Seed-paired comparisons . . . 17
11 Classification of claims by strength of evidence . . . 22
12 Limitations and mitigation . . . 23
13 Fields of the evaluation dataset . . . 27
14 Dataset integrity checks . . . 27
15 Preliminary versus recomputed values . . . 28

1 Introduction

Landing a vehicle autonomously is a constrained control problem: the controller must bring the vehicle to rest at a designated site with low touchdown velocity and small attitude error, while spending as little propellant as possible and never violating safety limits on the way down. Classical guidance solves this problem by designing a control law from a physical model. Reinforcement learning (RL) takes a different route. An agent learns a control policy by trial and error, guided only by a scalar reward signal that the engineer defines [1]. In RL the reward function therefore acts as the specification of the task: the agent is optimized to maximize the reward, not to satisfy the intentions behind it.

This distinction matters because a reward function is almost always an imperfect encoding of what the designer wants. When an optimizer finds a way to obtain high reward that the designer did not anticipate, the result is known as *specification gaming* [15] or *reward hacking* [14, 16]. A frequently cited early example comes from reward shaping. In a bicycle-riding task, a bonus for progress toward the goal led an agent to ride in small circles, collecting the progress bonus repeatedly instead of reaching the destination [13, 12]. The failure was not in the learning algorithm; the algorithm optimized exactly what it was given.

The present project reproduces this class of failure in a controlled setting and studies its correction. Four PPO agents [2] were trained in the Gymnasium LunarLander-v3 environment [4, 5] using Stable-Baselines3 [7]. The agents differ only in the reward used during training: the original reward, a small per-step time penalty, a deliberately flawed bonus for a "hover-like" state, and a corrected stability term. All four were then evaluated on identical starting conditions using the original, unmodified reward, so that the evaluation score measures the landing task itself rather than the shaped objective each agent was trained on.

This report is an independent technical analysis of that experiment. Its specific contributions are:

- a verification of every summary statistic against the raw evaluation records (400 episodes), including integrity checks on the dataset;
- an expanded statistical analysis with confidence intervals, seed-paired comparisons, and outcome classification;
- a mathematical analysis of the four reward definitions that explains why the flawed bonus creates an incentive to prolong episodes and why the corrected term removes that particular incentive;
- an explicit separation of verified observations, engineering interpretations, and hypotheses that would require further experiments.

Section 2 states the research questions. Sections 3–6 develop the theory and describe the environment. Sections 7–9 document the experimental design. Sections 10–13 present and analyze the results, and Sections 14–17 discuss their meaning, limitations, and the work needed to strengthen them.

2 Project Objectives and Research Questions

The project had five engineering objectives:

O1. Train an autonomous landing policy with PPO and establish its performance as a reference.

O2. Implement three alternative reward functions as Gymnasium wrappers, holding the algorithm, network, and training budget fixed.

O3. Evaluate all policies under a common protocol: the same seeds, deterministic actions, and the unmodified environment reward.

O4. Measure both task performance (score, landing rate, duration) and a behavioral indicator of the incentive under study (time spent slow, upright, and airborne).

O5. Characterize the mechanism of any reward misalignment and assess whether a redesigned reward recovers performance.

These objectives lead to four research questions, summarized in Table 1. The design expectations listed in the table were the stated intent of each reward variant. They were not pre-registered statistical hypotheses, and this report treats them as predictions to be compared with the evidence rather than as conclusions.

Table 1. Research questions, design expectations, and the evidence used to address them.

	Research question	Design expectation	Primary evidence
RQ1	How well does PPO with the unmodified reward land under the evaluation protocol?	Reliable landing; reference performance	Score, landing rate, outcome classes
RQ2	Does a -0.01 per-step penalty shorten episodes without degrading landing?	Shorter, equally successful episodes	Duration and landing rate vs. RQ1
RQ3	Does a $+1$ per-step bonus for a slow, upright state induce reward hacking?	Prolonged slow flight; fewer landings	Landings, time-limit endings, slow/upright frames
RQ4	Does a change-based stability reward recover landing performance?	Recovery toward baseline	Paired comparison with Agents 1 and 3

3 Theoretical Background: Reinforcement Learning

3.1 Markov decision processes

Reinforcement learning formalizes sequential decision-making as a Markov decision process (MDP) with state space $\mathcal{S}$, action space $\mathcal{A}$, transition dynamics $P(s_{t+1} \mid s_t, a_t)$, reward function R, and discount factor $0 \leq \gamma < 1$ [1]. At each time step t the agent observes a state s_t, selects an action a_t from its policy $\pi(a \mid s)$, and receives a reward r_{t+1} and the next state s_{t+1} (Figure 1). The discounted return from time t in an episode that ends at step T is

$$G_t \;=\; \sum_{k=0}^{T-t-1} \gamma^k \, r_{t+k+1}, \tag{1}$$

and the learning objective is to find policy parameters θ that maximize the expected return from the initial-state distribution,

$$J(\theta) \;=\; \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \right]. \tag{2}$$

The state-value function $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ and the action-value function $Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ define the advantage $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$, which measures how much better an action is than the policy's average behavior in that state.

The discount factor sets an effective planning horizon of roughly $1/(1-\gamma)$ steps. With $\gamma = 0.99$, the value used in this study (Section 4), rewards more than a few hundred steps ahead carry little weight. This horizon becomes important in Section 12.

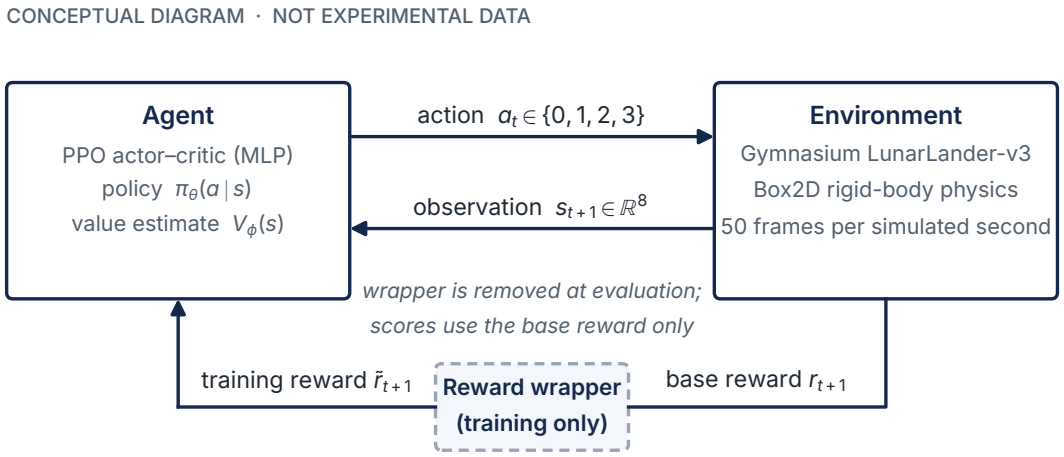


Figure 1. Agent–environment interaction in this study (conceptual diagram). The reward wrapper modifies the environment reward only during training; all evaluation scores use the unmodified base reward.

3.2 Termination and truncation

An episode can end in two ways that must be treated differently. *Termination* means the task has genuinely reached an end state, such as a crash or a completed landing; no further reward follows. *Truncation* means the episode was cut off by an external time limit even though the underlying process could continue. Pardo et al. [17] show that treating a time limit as a true terminal state makes the problem non-Markovian unless the remaining time is part of the observation, and recommend bootstrapping from the value of the last state instead. Stable-Baselines3 follows this recommendation for truncated episodes [10]. The consequence for this study is that, from the learner's perspective, a state that can be maintained indefinitely is valued as if its rewards continue beyond the time limit.

3.3 Reward shaping

Reward shaping adds a supplementary term F to the environment reward to make learning faster or more reliable:

$$\tilde{r}_{t+1} \;=\; r_{t+1} + F(s_t, a_t, s_{t+1}). \tag{3}$$

Ng, Harada, and Russell [12] proved that shaping of the *potential-based* form

$$F(s, s') \;=\; \gamma\,\Phi(s') - \Phi(s), \tag{4}$$

for an arbitrary real-valued potential Φ over states, leaves the set of optimal policies unchanged; any shaping term not of this form can, in general, change which policy is optimal. The bicycle example in Section 1 is such a case: a bonus that can be collected repeatedly around a cycle of states makes cycling more valuable than finishing. Equation (4) is the reference against which the corrected reward of Section 8 is assessed.

3.4 Specification gaming and reward hacking

Krakovna et al. [15] define specification gaming as behavior that satisfies the literal specification of an objective without achieving the outcome the designer intended. Skalse et al. [16] formalize a closely related notion: a proxy reward is *hackable* relative to a true reward if increasing the expected proxy return can decrease the expected true return. In this study, the true reward is the unmodified LunarLander reward, the proxies are the four training rewards, and the evaluation protocol measures the true reward directly.

4 Proximal Policy Optimization and Neural Network Architecture

4.1 Clipped policy optimization

Policy-gradient methods improve $J(\theta)$ by gradient ascent using sampled trajectories. A known difficulty is step size: a single large update can move the policy into a region where its own data no longer describe its behavior, and performance can collapse. PPO [2] limits the change in the policy at each update. With the probability ratio

$$\rho_t(\theta) \;=\; \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \tag{5}$$

PPO maximizes the clipped surrogate objective

$$L^{\text{CLIP}}(\theta) \;=\; \hat{\mathbb{E}}_t\Big[\min\Big(\rho_t(\theta)\,\hat{A}_t,\; \text{clip}\left(\rho_t(\theta),\, 1-\epsilon,\, 1+\epsilon\right)\hat{A}_t\Big)\Big], \tag{6}$$

where $\hat{A}_t$ is an advantage estimate and ϵ is the clip range. The minimum makes the objective a pessimistic bound: the policy gains nothing by moving the ratio beyond $1 \pm \epsilon$ in the direction that the advantage favors.

Advantages are computed with generalized advantage estimation (GAE) [3], which forms an exponentially weighted sum of temporal-difference residuals:

$$\delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \qquad \hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l}. \tag{7}$$

The parameter $\lambda \in [0,1]$ trades bias for variance. The full objective adds a value-function regression loss L^{VF} and an entropy bonus $\mathcal{H}$ that encourages exploration:

$$L(\theta,\phi) \;=\; L^{\text{CLIP}}(\theta) \;-\; c_1\, L^{\text{VF}}(\phi) \;+\; c_2\, \mathcal{H}\big[\pi_\theta\big]. \tag{8}$$

4.2 Implementation and hyperparameters

The agents were trained with the Stable-Baselines3 implementation of PPO [7, 8]. The project documentation does not record any hyperparameter overrides, so the library defaults listed in Table 2 are assumed. These values are documented software defaults, not values read from a saved configuration or training log; Section 15 treats this as a reproducibility gap.

Table 2. PPO hyperparameters assumed for all four agents (Stable-Baselines3 defaults [8]; not independently logged).

Parameter	Value	Role
Rollout length n_{steps}	2,048	Environment steps collected before each update
Minibatch size	64	Samples per gradient step
Epochs per update	10	Passes over each rollout
Learning rate (Adam)	3×10^{-4}	Optimizer step size
Discount factor γ	0.99	Weight of future rewards, Eq. (1)
GAE parameter λ	0.95	Bias–variance trade-off, Eq. (7)
Clip range ϵ	0.2	Trust-region width, Eq. (6)
Value-loss coefficient c_1	0.5	Weight of L^{VF}, Eq. (8)
Entropy coefficient c_2	0.0	No explicit entropy bonus
Gradient-norm limit	0.5	Gradient clipping
Advantage normalization	enabled	Per-minibatch standardization

4.3 Policy and value networks

The `MlpPolicy` of Stable-Baselines3 is an actor–critic architecture. For a one-dimensional observation, the default is a fully connected network with two hidden layers of 64 units, tanh activations, and orthogonal weight initialization [9, 8]. In current library versions the actor and the critic each have their own pair of hidden layers. The actor maps the eight-dimensional observation to four logits that define a categorical distribution over actions; the critic maps the same observation to a scalar value estimate $V_\phi(s)$. Under these defaults the actor has $8 \cdot 64 + 64 + 64 \cdot 64 + 64 + 64 \cdot 4 + 4 = 4{,}996$ parameters and the critic 4,801, for 9,797 trainable parameters in total. This count follows from the library defaults and was not verified against the saved model files, which were not available.

During training, actions are sampled from the categorical distribution, which provides exploration. During evaluation, the most probable action is selected (`deterministic=True`). The evaluated controller is therefore a deterministic function of the observation, not the stochastic policy that generated the training data. Figure 2 summarizes the complete training and evaluation workflow.

5 LunarLander Simulation Environment

LunarLander is a two-dimensional rigid-body simulation built on the Box2D physics engine, originally written by Oleg Klimov and maintained in the Gymnasium library by the Farama Foundation [4, 5]. A lander starts near the top center of the scene with a random initial force applied to its center of mass. The terrain is randomly generated except for a flat landing pad centered at the origin. The simulation advances at 50 frames per second, and each environment step is one frame.

This study uses the discrete-action version, LunarLander-v3, which is registered with a time limit of 1,000 steps (20 s of simulated time) and a reward threshold of 200 points [6]. Version 3 fixed a source of non-determinism caused by incomplete destruction of the physics world between episodes and widened the observation bounds [5]. The environment options for continuous control, wind, and gravity were not recorded; their defaults (discrete actions, no wind, gravity of -10 in simulation units) are assumed.

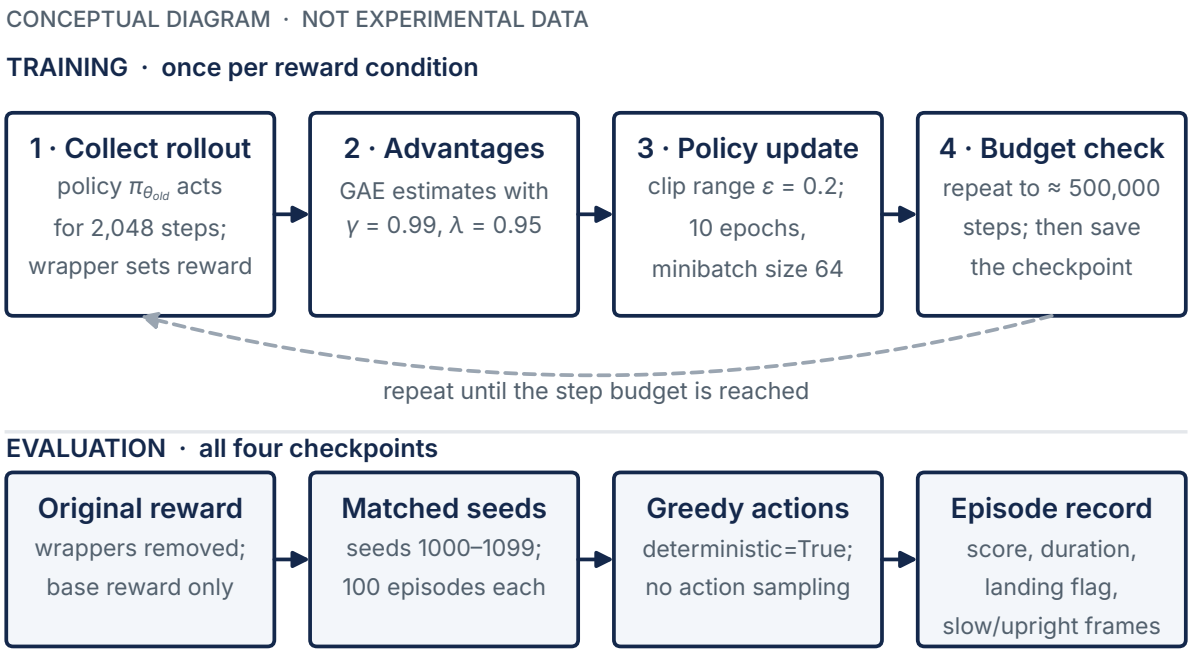


Figure 2. PPO training loop and evaluation protocol used for the four agents (conceptual diagram). Hyperparameter values are the Stable-Baselines3 defaults assumed in Table 2.

An episode terminates when (i) the lander body touches the ground (a crash), (ii) the lander's horizontal position leaves the viewport ($|x| \geq 1$ in normalized coordinates), or (iii) the lander comes to rest and the physics engine puts the body to sleep. If none of these occurs, the episode is truncated at step 1,000.

The simulation is a benchmark for learning algorithms, not a model of a real lunar vehicle. Its gravity, thrust, and length scales are not calibrated to lunar conditions, and its units are simulation units rather than SI units. Conclusions drawn here concern learning behavior and reward design, not spacecraft performance.

6 State Space, Action Space, and Reward Mechanism

6.1 Observation and action spaces

The agent observes the eight-dimensional vector

$$s_t = \left(x,\ y,\ v_x,\ v_y,\ \theta,\ \omega,\ c_L,\ c_R\right)_t, \tag{9}$$

whose components are listed in Table 3. Positions and velocities are normalized by the viewport geometry and frame rate. In particular, the observed velocities are $v_x = 0.2\,\dot{x}$ and $v_y \approx 0.133\,\dot{y}$, where $\dot{x}$ and $\dot{y}$ are velocities in simulation units per second (Appendix C). The attitude angle θ is in radians. The observation does not include the elapsed time.

The action space has four discrete actions: 0, no engine; 1, fire the left orientation engine; 2, fire the main engine; and 3, fire the right orientation engine. Only one engine can be commanded per step, so throttle is effectively modulated by switching the main engine on and off from frame to frame.

Table 3. Observation vector of LunarLander-v3 [5, 6].

Index	Symbol	Meaning
0, 1	x, y	Horizontal and vertical position relative to the landing pad, normalized by half the viewport width and height
2, 3	v_x, v_y	Horizontal and vertical velocity, normalized by viewport geometry and frame rate
4	θ	Body angle (rad); $\theta = 0$ is upright
5	ω	Angular velocity, scaled by 20/FPS
6, 7	c_L, c_R	Left and right leg ground contact (1 = contact, 0 = no contact)

6.2 Base reward

The reference implementation defines a shaping potential over states [6],

$$\Psi(s) = -100\sqrt{x^2 + y^2} \; - \; 100\sqrt{v_x^2 + v_y^2} \; - \; 100\,|\theta| \; + \; 10\,c_L \; + \; 10\,c_R, \tag{10}$$

and computes the per-step base reward as the change in this potential minus engine costs:

$$r_{t+1} = \begin{cases} -100 & \text{if the lander crashes or leaves the viewport,} \\ +100 & \text{if the lander comes to rest,} \\ \Psi(s_{t+1}) - \Psi(s_t) - 0.30\,m_t - 0.03\,e_t & \text{otherwise,} \end{cases} \tag{11}$$

where $m_t \in \{0, 1\}$ indicates main-engine firing and $e_t \in \{0, 1\}$ side-engine firing in the discrete version. In the implementation, the terminal value replaces the shaping term of the final step rather than being added to it.

Two properties of Eq. (11) are used later. First, apart from the final step, the shaping terms telescope: over an episode they sum to the net change in Ψ, so the base return is approximately the improvement in position, speed, and attitude, minus total engine cost, plus the terminal ± 100. Second, the reward is not discounted inside Ψ, so the base reward is not itself exactly of the form of Eq. (4). An agent that holds a fixed state earns zero shaping reward and pays 0.30 per frame of main-engine use. A hovering agent therefore accumulates a negative base return, which is relevant to the hover-reward results in Section 12.

7 Experimental Setup and Training Methodology

7.1 Software and hardware

Table 4 lists the software stack. The project documentation names the libraries but does not record their exact versions; LunarLander-v3 implies Gymnasium 1.0 or later.

7.2 Training procedure

Each reward condition was trained once, in a separate run, with the requested budget of 500,000 environment steps. Because PPO updates only after a complete rollout, the actual number of steps is rounded up to a multiple of the rollout length. The project documentation reports 501,760

Table 4. Software environment and what was recorded.

Component	Specification	Version recorded?
Language	Python 3.12	Minor version only
Environment	Gymnasium, LunarLander-v3 (Box2D)	No (v3 requires Gymnasium ≥ 1.0)
RL library	Stable-Baselines3, PPO with `MlpPolicy`	No
Deep-learning backend	PyTorch	No
Reward modification	Custom Gymnasium reward wrappers	Source not supplied
Compute	CPU training (per project documentation)	No hardware details

steps for the time-penalty run, which equals $245 \times 2{,}048$ and is consistent with a single environment and the default rollout length of 2,048 steps; the training log itself is not part of the supplied material. According to the documentation, the training random seeds were not fixed, and the four runs saved checkpoints named `lunar_lander_ppo.zip`, `lunar_lander_shaped.zip`, `lunar_lander_hover.zip`, and `lunar_lander_fixed.zip`. Neither the checkpoints nor the training scripts were included in the package examined for this report.

Each modified reward was applied by wrapping the training environment in a Gymnasium reward wrapper, which intercepts the reward returned by `step()` and returns the modified value $\tilde{r}$ to the learner. The wrapper does not change the dynamics, the observation, or the termination rules.

7.3 Provenance of the evidence

The primary experimental record is `final_results.csv`: 400 rows, one per evaluation episode, with seven fields (Appendix A). The supplied files `summary_statistics.csv` and `paired_comparisons.csv` were derived from it in an earlier analysis. For this report, all statistics were recomputed from the raw file with new scripts (Appendix D), and the earlier derived files were used only as a cross-check. No training curves, videos, trajectory logs, or model files were available, and none are presented or implied.

8 Design of the Four Reward Functions

Every agent was trained on a reward of the form $\tilde{r}_{t+1} = r_{t+1} + F_{t+1}$, where r_{t+1} is the base reward of Eq. (11) and F_{t+1} is the added term. Table 5 summarizes the four designs. The analysis that follows is derived from the reward definitions; it describes incentives, not measured behavior.

Table 5. The four reward designs. Each agent was trained on $\tilde{r} = r + F$ and evaluated on r.

Agent	Added term F_{t+1}	Purpose
1 Baseline	0	Reference performance with the original reward
2 Time penalty	-0.01 per step	Test whether penalizing duration improves efficiency
3 Hover reward	$+1$ per eligible step, Eq. (13)	Deliberately flawed incentive to study reward hacking
4 Corrected reward	$2\,\Delta S - 0.01$, Eq. (16)	Reward stability improvement, not stability itself

8.1 Agent 1: baseline

Agent 1 uses the unmodified reward, $\tilde{r}_{t+1} = r_{t+1}$. It provides the reference against which the other designs are judged.

8.2 Agent 2: time penalty

Agent 2 subtracts a constant at every step:

$$\tilde{r}_{t+1} = r_{t+1} - 0.01. \tag{12}$$

The intent is to make long episodes costlier and so encourage faster landings. The magnitude of the term is small compared with the existing reward: it is one-thirtieth of the 0.30 cost of a single frame of main-engine firing, amounts to -3 points over a typical 300-step episode, and has a discounted value of at most $-0.01/(1-\gamma) = -1$ point with $\gamma = 0.99$. By comparison, the terminal outcome alone is worth ± 100 points. On this analysis the penalty is expected to have a weak effect on the optimal policy.

8.3 Agent 3: hover reward

Agent 3 adds a bonus of one point on every step at which the lander is slow and nearly upright:

$$\tilde{r}_{t+1} = r_{t+1} + \mathbf{1}\{E(s_{t+1})\}, \tag{13}$$

where the eligibility condition is

$$E(s) \;\equiv\; |v_x| < 0.2 \;\wedge\; |v_y| < 0.2 \;\wedge\; |\theta| < 0.15 \;\wedge\; \text{episode not ended}. \tag{14}$$

In simulation units, the thresholds correspond to a horizontal speed below 1.0 unit/s, a vertical speed below 1.5 units/s, and a tilt below 0.15 rad (8.6°). As specified, the condition does not require the legs to be off the ground.

The design is flawed by construction. The bonus is paid for *being* in a desirable-looking state, and it is paid again on every step that the state persists. A landing, by contrast, is rewarded once and ends the episode, which also ends the stream of bonuses. Holding the eligible state for n steps adds n points to the training return, up to 999 points in a 1,000-step episode, an order of magnitude more than the $+100$ landing reward. Equation (13) is not of the potential-based form of Eq. (4), so it can change the optimal policy. Section 12 examines the resulting incentive in detail.

8.4 Agent 4: corrected reward

Agent 4 replaces the state bonus with a reward for *improving* stability. Define the stability measure

$$S(s) \;=\; -\big(|v_x| + |v_y| + |\theta|\big), \tag{15}$$

which is at most zero and equals zero only for a motionless, upright lander. The training reward is

$$\tilde{r}_{t+1} \;=\; r_{t+1} \;+\; 2\big[S(s_{t+1}) - S(s_t)\big] \;-\; 0.01. \tag{16}$$

Summed over an episode of T steps, the added term telescopes:

$$\sum_{t=0}^{T-1} F_{t+1} \;=\; 2\big[S(s_T) - S(s_0)\big] \;-\; 0.01\,T. \tag{17}$$

The undiscounted bonus therefore depends only on the stability of the first and last states and on the episode length. Holding any fixed state earns nothing from the stability term and costs 0.01 per step, so the incentive to remain in the eligible state indefinitely is removed. For a fixed starting state, an episode that ends at rest ($S(s_T) \approx 0$) after few steps collects more of this bonus than one that hovers until the time limit.

Equation (16) is close to, but not exactly, potential-based shaping. Writing the stability term with potential $\Phi = 2S$,

$$2\big[S(s_{t+1}) - S(s_t)\big] = \underbrace{2\big[\gamma S(s_{t+1}) - S(s_t)\big]}_{\text{potential-based, Eq. (4)}} + \underbrace{2(1-\gamma)\,S(s_{t+1})}_{\text{residual}}, \tag{18}$$

shows that the design adds a small residual per-step penalty, $0.02\,S(s_{t+1}) \leq 0$ for $\gamma = 0.99$, for instability. Together with the -0.01 time cost, this residual means the policy-invariance guarantee of Ng et al. [12] does not formally apply. The corrected design should therefore be viewed as a heuristic that removes the specific exploit of Agent 3, not as a provably neutral shaping term. How the wrapper initialized $S(s_t)$ on the first step after a reset is not documented.

9 Evaluation Methodology and Performance Metrics

9.1 Protocol

Each of the four saved policies was evaluated for 100 episodes. Episode i was started with the environment reset seed $1000 + i - 1$, so every agent faced the same 100 initial conditions (seeds 1000 to 1099). Actions were selected deterministically. The reward wrappers were removed, so every score is the sum of unmodified base rewards from Eq. (11). A consistency check supports this: the highest evaluation score of the hover agent is 2.2 points, whereas scores that included its training bonus would be expected to reach several hundred points.

9.2 Metrics

For each episode i the dataset records the following quantities.

Score. $R_i = \sum_t r_{t+1}$, the undiscounted return under the original reward. The conventional reference for a "solved" episode is $R_i \geq 200$ [5].

Duration. d_i, the number of steps until termination or truncation ($d_i \leq 1{,}000$).

Landing flag. $\ell_i = 1$ if the episode terminated with a final base reward of at least $+100$. Under Eq. (11) this occurs exactly when the lander comes to rest. The flag does not check whether the lander rested on the pad, and the environment allows landing outside the pad [6].

Slow/upright airborne frames. h_i, the number of non-terminal steps with $|v_x| < 0.2$, $|v_y| < 0.2$, $|\theta| < 0.15$, and both legs off the ground ($c_L = c_R = 0$). This indicator is narrower than the training condition of Eq. (13) because it requires the lander to be airborne.

Two derived quantities are used. The per-episode share of slow/upright airborne frames is $f_i = h_i/d_i$, and the pooled share for an agent is $\sum_i h_i / \sum_i d_i$. The mean of f_i weights every episode equally, whereas the pooled share weights every frame equally; long episodes therefore count more in the pooled share.

Each episode is also assigned one of three outcome classes: *landed* ($\ell_i = 1$); *time limit reached* ($d_i = 1{,}000$); and *terminated without landing* (all other episodes). Under the environment's termination

rules (Section 5), the last class contains crashes and exits from the viewport, which the dataset cannot distinguish.

The slow/upright indicator does not identify hovering in the sense of holding a fixed position. A lander descending slowly and steadily toward the pad also satisfies it. The indicator measures time spent in the state the hover bonus rewarded, and it must be interpreted together with duration and outcome.

9.3 Statistical methods

Each episode is one observation, so $n = 100$ per agent. Means are reported with standard deviations (divisor $n - 1$) and 95% percentile-bootstrap confidence intervals from 10,000 resamples with a fixed random seed [20]; durations use the t-interval. Medians and interquartile ranges describe the distributions, which are often bimodal. Landing rates use the Wilson score interval [21],

$$\frac{\hat{p} + \dfrac{z^2}{2n}}{1 + \dfrac{z^2}{n}} \pm \frac{z}{1 + \dfrac{z^2}{n}} \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}, \qquad z = 1.96, \tag{19}$$

which remains valid near $\hat{p} = 0$, where the familiar normal-approximation interval fails.

Because all agents faced the same seeds, comparisons between agents are paired by seed. Paired score differences are summarized with bootstrap intervals and tested with the Wilcoxon signed-rank test [22]; paired landing outcomes are tested with the exact McNemar test [23], which uses only the seeds on which the two agents disagree. The six pairwise comparisons in each family are adjusted with the Holm procedure [24].

> **Scope of the statistical inference**
>
> All intervals and tests in this report treat the 100 evaluation episodes as a sample of starting conditions for a fixed, already-trained policy. They answer the question "how would this particular checkpoint perform on other starting conditions?" They do not answer "how would a new PPO agent trained with this reward perform?", because each reward was trained only once and training outcomes vary substantially between random seeds [18, 19]. Answering the second question requires repeated training (Section 16).

10 Experimental Results

This section reports verified observations only. All values were recomputed from the raw evaluation dataset. The four summary values supplied with the project for each agent (mean score, landing rate, and mean slow/upright frames) were reproduced exactly (Appendix E).

10.1 Task performance

Table 6 and Figure 3 summarize the evaluation scores. The baseline obtained the highest mean score, 177.5 points, with a median of 227.7 points; 68 of its 100 episodes reached the 200-point reference. The corrected agent ranked second (mean 90.2, median 150.3). The time-penalty agent (mean −15.0) and the hover agent (mean −60.6) both had negative mean scores. The hover agent's scores were the least variable (SD 35.6), and none was above 2.2 points.

Table 6. Evaluation scores under the unmodified reward (100 episodes per agent). CI: 95% percentile bootstrap. IQR: interquartile range. ≥200: episodes scoring at least 200 points.

Agent	Mean	95% CI of mean	SD	Median	IQR	Min	Max	≥200
1 Baseline	177.53	[156.9, 197.0]	103.7	227.7	144.4 to 247.6	−42.9	286.5	68
2 Time penalty	−14.98	[−40.3, 11.3]	130.1	−73.4	−100.9 to 147.2	−214.1	228.9	9
3 Hover reward	−60.63	[−67.9, −54.2]	35.6	−53.9	−74.0 to −40.9	−277.2	2.2	0
4 Corrected reward	90.21	[65.3, 114.5]	126.5	150.3	−40.4 to 207.3	−147.2	242.5	27

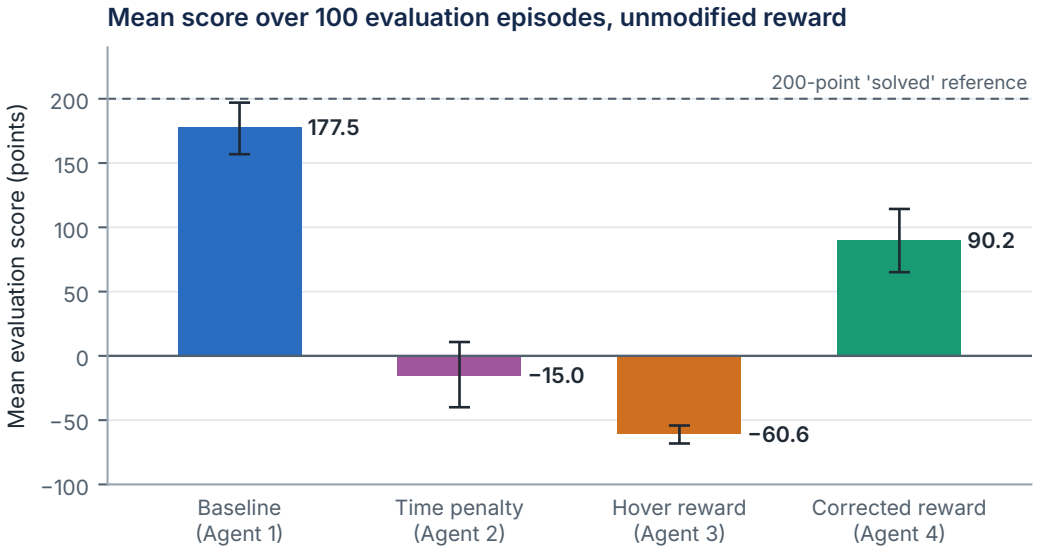


Figure 3. Mean evaluation score per agent with 95% bootstrap confidence intervals (measured data, 100 episodes each). The dashed line marks the 200-point reference for a solved episode.

10.2 Landing outcomes

Figure 4 and Table 7 show how each episode ended. The baseline came to rest in 75 episodes (Wilson 95% CI 65.7% to 82.5%), the corrected agent in 55 (45.2% to 64.4%), the time-penalty agent in 27 (19.3% to 36.4%), and the hover agent in none (0% to 3.7%). The failure modes differ markedly. The baseline and time-penalty agents failed almost entirely by early termination (24 and 72 episodes), with one time-limit episode each. The hover agent reached the 1,000-step limit in 87 of 100 episodes. The corrected agent combined 36 early terminations with 9 time-limit episodes.

Table 7. Episode outcomes and durations. Crash/OOB: terminated without landing (crash or out of bounds, not separable in the data). Durations in steps (50 steps = 1 s of simulated time); CI is the 95% t-interval of the mean.

Agent	Landed	95% Wilson CI (%)	Crash/OOB	Time limit	Mean dur.	95% CI	Median dur.
1 Baseline	75/100	[65.7, 82.5]	24	1	301.9	[279, 325]	275.5
2 Time penalty	27/100	[19.3, 36.4]	72	1	382.4	[349, 416]	375.0
3 Hover reward	0/100	[0.0, 3.7]	13	87	931.9	[892, 972]	1,000.0
4 Corrected reward	55/100	[45.2, 64.4]	36	9	503.8	[453, 555]	400.5

Table 8 separates the episodes that landed from those that did not. Successful landings by the time-penalty and corrected agents took far longer than the baseline's: a mean of 533 and 513 steps,

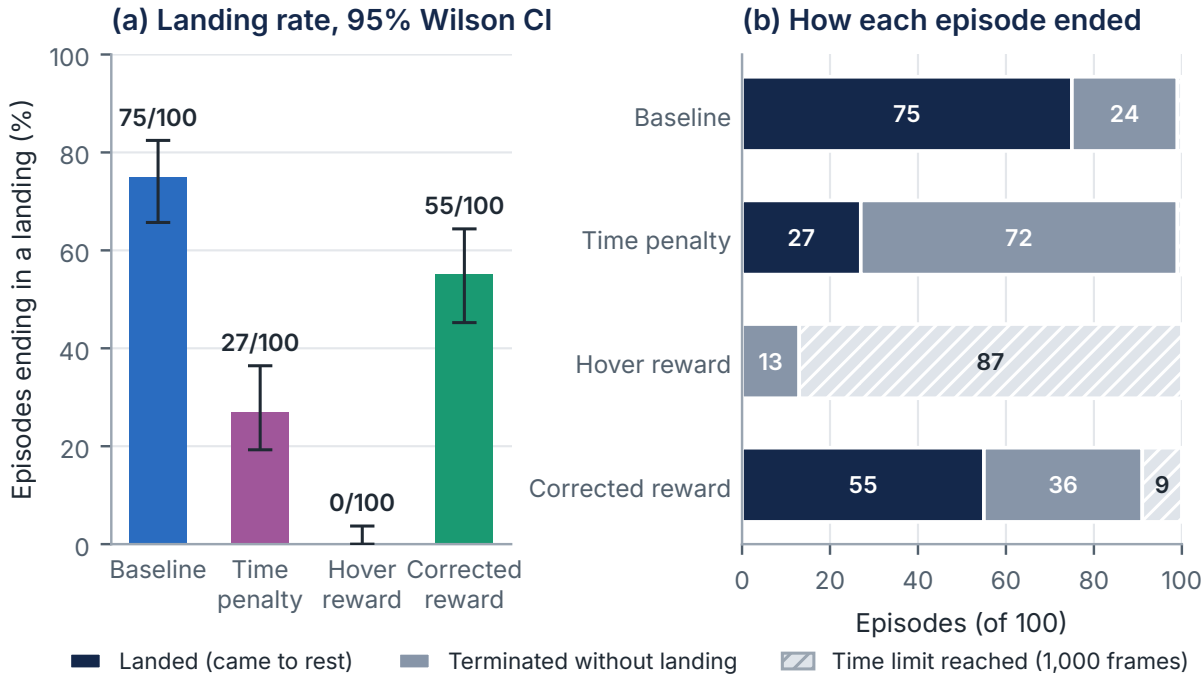


Figure 4. (a) Landing rate with 95% Wilson confidence intervals. (b) Outcome of every evaluation episode: came to rest, terminated without landing (crash or exit from the viewport), or reached the 1,000-step time limit (measured data).

compared with 315 steps. The mean score of a successful landing was also lower for these agents (190.0 and 197.2 points, compared with 235.0).

Table 8. Mean score (points) and mean duration (steps) conditioned on whether the episode ended in a landing.

Agent	Score, landed	Score, not landed	Duration, landed	Duration, not landed
1 Baseline	235.0	5.1	315	263
2 Time penalty	190.0	−90.8	533	327
3 Hover reward	n/a	−60.6	n/a	932
4 Corrected reward	197.2	−40.6	513	492

11 Statistical and Graphical Analysis

11.1 Distribution of scores

The mean scores in Figure 3 conceal strongly bimodal distributions. Figure 5 plots every episode. For the baseline, time-penalty, and corrected agents, scores fall into a cluster near or above 200 points (landings) and a second cluster near or below zero (failures), with almost nothing in between. The time-penalty agent's median (−73.4) lies in its failure cluster, while its mean (−15.0) falls in the gap between clusters, where few episodes actually occur. For distributions of this shape, the landing rate and the median are more informative summaries than the mean. The hover agent's scores, by contrast, form a single narrow cluster around −55 points.

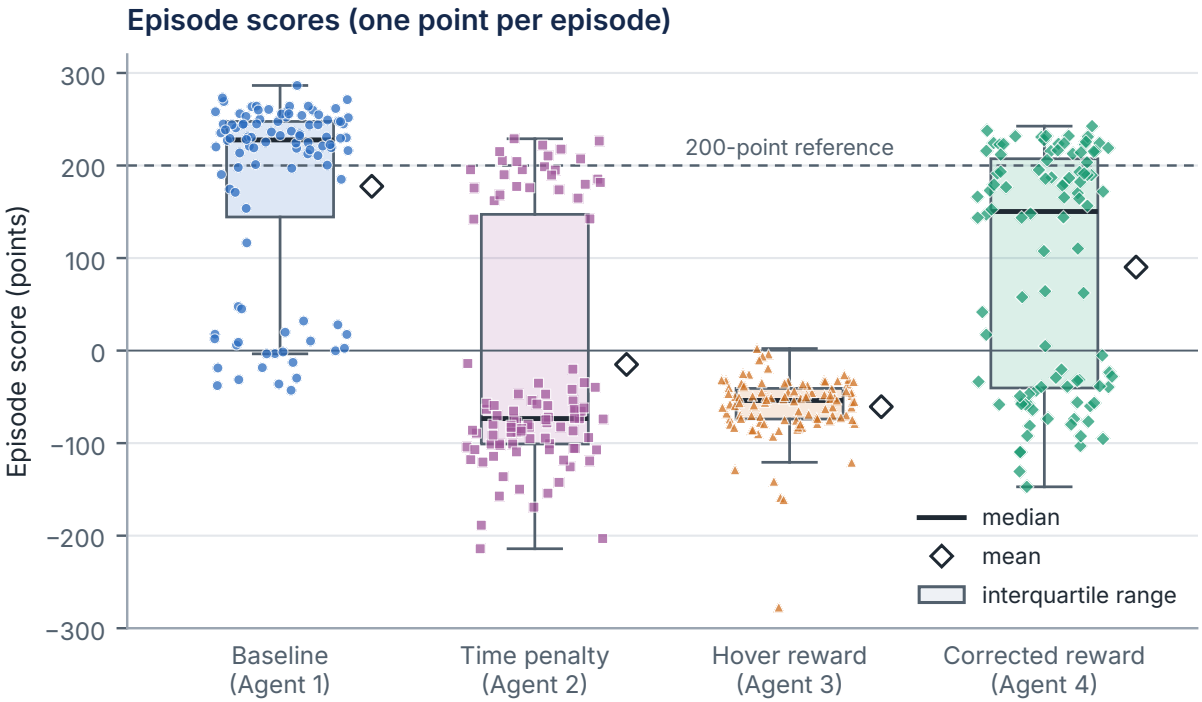


Figure 5. Distribution of episode scores (measured data). Each marker is one evaluation episode; boxes show the interquartile range, the thick line the median, and the open diamond the mean. Whiskers extend to the most extreme point within 1.5 IQR of the box.

11.2 Episode duration

Figure 6 shows that the reward condition changed episode length substantially. Mean duration rose from 302 steps for the baseline to 382 steps with the time penalty, 504 steps with the corrected reward, and 932 steps with the hover reward, whose median episode lasted the full 1,000 steps. The time-penalty result runs opposite to the design expectation of Table 1: the agent penalized for duration took longer than the baseline, by 80.5 steps on average over the same seeds (95% CI 41.4 to 120.2).

11.3 Slow, upright, airborne behavior

Figure 7 and Table 9 report the behavioral indicator. The hover agent spent a mean of 842.4 frames per episode in the slow/upright airborne state, 86.6% of its average episode, and in 90 of its 100 episodes the share exceeded 75%. The baseline spent 28.1 frames (8.9%) in this state. The time-penalty and corrected agents were intermediate, at 30.7% and 43.3%. Pooled shares are higher than per-episode means for every agent because long episodes contain proportionally more slow/upright frames.

The indicator must be read together with the outcome. In the episodes that ended in a landing, the time-penalty and corrected agents spent about half of their frames in the slow/upright airborne state, compared with about a tenth for the baseline. Since these episodes ended at rest, much of that time must correspond to slow, controlled flight that eventually reached the ground rather than to indefinite hovering. High occupancy of the state is therefore not by itself evidence of reward hacking; it becomes so when combined with the absence of landings and with episodes that run to the time limit.

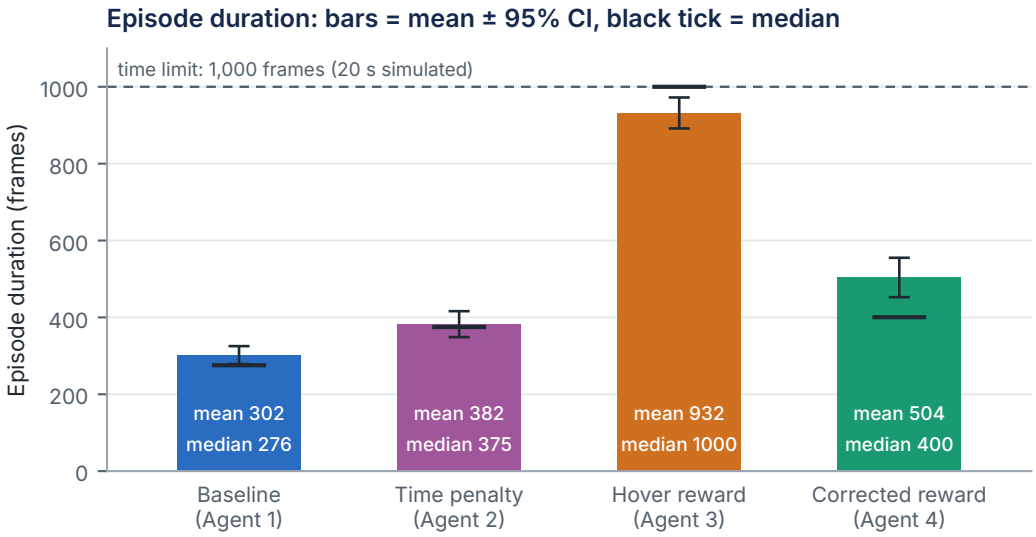


Figure 6. Episode duration per agent: mean with 95% confidence interval (bar and error bar) and median (black tick), in steps (measured data). The dashed line is the 1,000-step time limit.

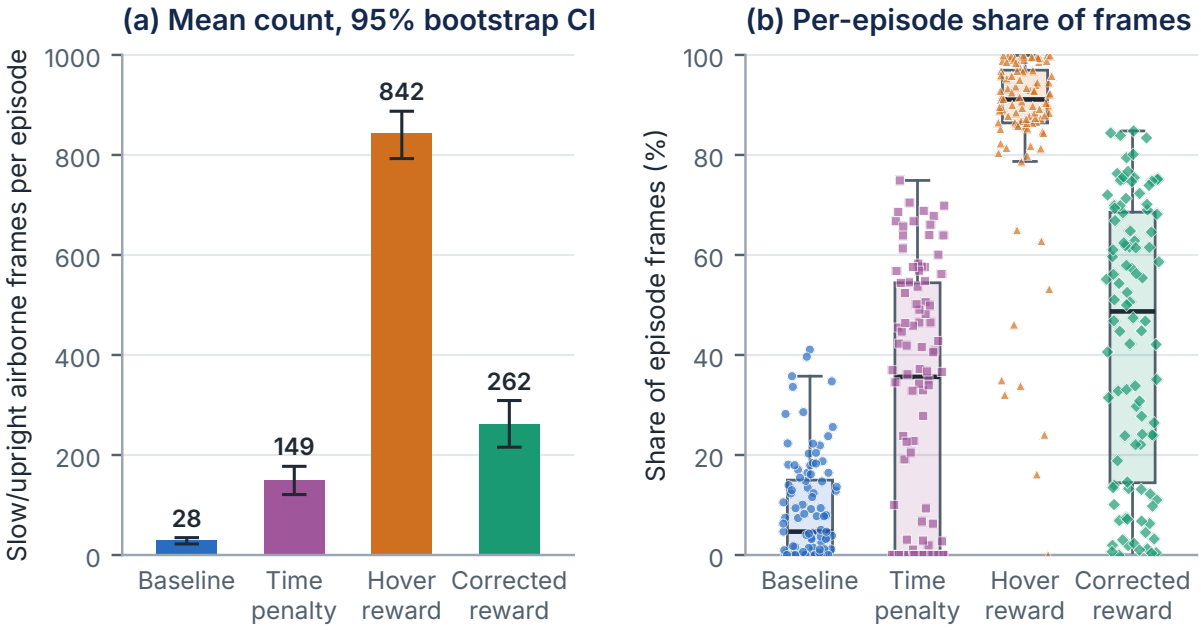


Figure 7. Slow/upright airborne behavior (measured data). (a) Mean number of qualifying frames per episode with 95% bootstrap intervals. (b) Per-episode share of frames in the state; each marker is one episode.

Table 9. Slow/upright airborne frames per episode. Mean share: mean of per-episode shares h_i/d_i. Pooled share: $\sum h_i / \sum d_i$.

Agent	Mean frames	SD	Median	Mean share (%)	Pooled share (%)	Episodes ≥75%
1 Baseline	28.14	33.0	14.5	8.9	9.3	0
2 Time penalty	148.82	147.2	150.5	30.7	38.9	0
3 Hover reward	842.38	242.6	911.0	86.6	90.4	90
4 Corrected reward	261.74	237.3	210.0	43.3	51.9	13

11.4 Duration and score

Figure 8 relates duration to score for every episode. For the baseline, most landings occur between about 250 and 400 steps, and failures are short. For the corrected agent, successful landings are spread across almost the whole range of durations, up to nearly 1,000 steps; long flight was not always followed by failure. For the hover agent, the time-limit episodes form a vertical band at 1,000 steps with scores between about -100 and 0, while its early terminations score lower. The rank correlations between duration and score are weak to moderate ($\rho = 0.11$ to 0.48), so episode length alone is a poor predictor of score.

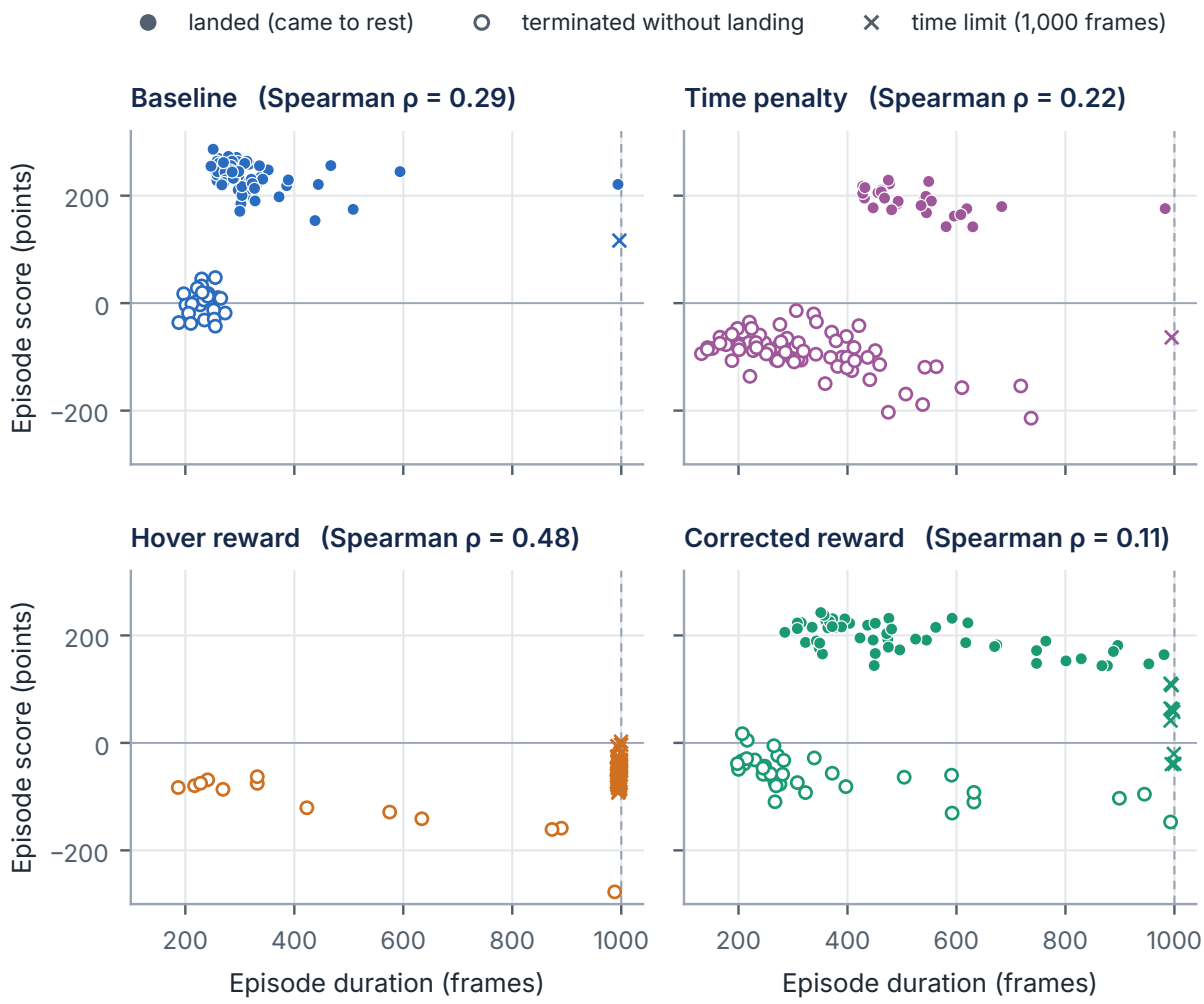


Figure 8. Episode duration against score for each agent, marked by outcome (measured data). Time-limit episodes are jittered slightly to the left of the 1,000-step line so that overlapping points remain visible. ρ is Spearman's rank correlation.

11.5 Paired comparisons

Table 10 compares the agents seed by seed. After Holm adjustment, five of the six score comparisons and all six landing comparisons differ at the 0.05 level. The exception is the score comparison between the hover and time-penalty agents. Their mean paired difference is -45.7 points with a bootstrap interval that excludes zero, yet the Wilcoxon test gives $p = 0.44$. The two results are not contradictory: they measure different things. The median paired difference is $+14.7$ points;

the hover agent scored higher on most seeds, but the time-penalty agent's landings, when they occurred, outscored the hover agent by a wide margin and dominated the mean.

Table 10. Seed-paired comparisons (difference = first agent minus second). Δ score: mean paired difference in points with 95% bootstrap CI; p_W: Wilcoxon signed-rank, Holm-adjusted. Δ landed: difference in landing rate (percentage points); Discordant: seeds on which only the first / only the second agent landed; p_M: exact McNemar, Holm-adjusted.

Comparison	Δ score	95% CI	p_W	Δ landed (pp)	95% CI	Discordant	p_M
Time pen. − Baseline	−192.5	[−219.7, −165.5]	< 0.001	−48	[−59, −38]	2 / 50	< 0.001
Hover − Baseline	−238.2	[−257.9, −217.6]	< 0.001	−75	[−83, −66]	0 / 75	< 0.001
Corrected − Baseline	−87.3	[−113.9, −61.9]	< 0.001	−20	[−31, −9]	8 / 28	0.001
Hover − Time pen.	−45.7	[−73.3, −20.3]	0.44	−27	[−36, −19]	0 / 27	< 0.001
Corrected − Time pen.	105.2	[76.2, 134.7]	< 0.001	28	[17, 39]	34 / 6	< 0.001
Corrected − Hover	150.8	[126.6, 174.4]	< 0.001	55	[45, 65]	55 / 0	< 0.001

Seeds are not equally difficult. Scores on the same seed are positively correlated across agents; for example, Spearman's $\rho = 0.40$ between the baseline and the corrected agent, and 0.50 between the baseline and the time-penalty agent. Pairing removes this shared component, which is why the paired intervals in Table 10 are narrower than a comparison of independent group means would give.

11.6 Uncertainty

Three sources of uncertainty affect these results, and the intervals capture only the first.

1. **Evaluation sampling.** With 100 episodes, a landing rate is known to within roughly ±9 to ±10 percentage points near 50%, and mean scores to within about ±20 to ±26 points for the three agents with bimodal score distributions. These intervals are reported throughout.
2. **Training variability.** A second training run with a different random seed could produce a substantially better or worse policy under the same reward. With one run per condition this variance cannot be estimated, so differences between agents mix the effect of the reward with the effect of the particular run.
3. **Design choices.** Each reward was tested at a single coefficient, with a single training budget and default hyperparameters. The results may not hold for other settings.

12 Reward Hacking: Mechanism and Observed Behavior

12.1 Mechanism (analysis)

Figure 9 contrasts the intended objective with the path the hover bonus makes attractive. Both start from the same slow, upright state that a good landing passes through. From that state, completing the landing earns +100 once and ends the episode. Remaining in the state earns +1 on every frame and keeps the episode going.

Figure 10 quantifies the incentive from the reward definitions alone. Without discounting, the added training reward from holding the state for n frames grows linearly to almost 1,000 points, ten times the landing reward. With the discount factor used by PPO, the value of the bonus stream

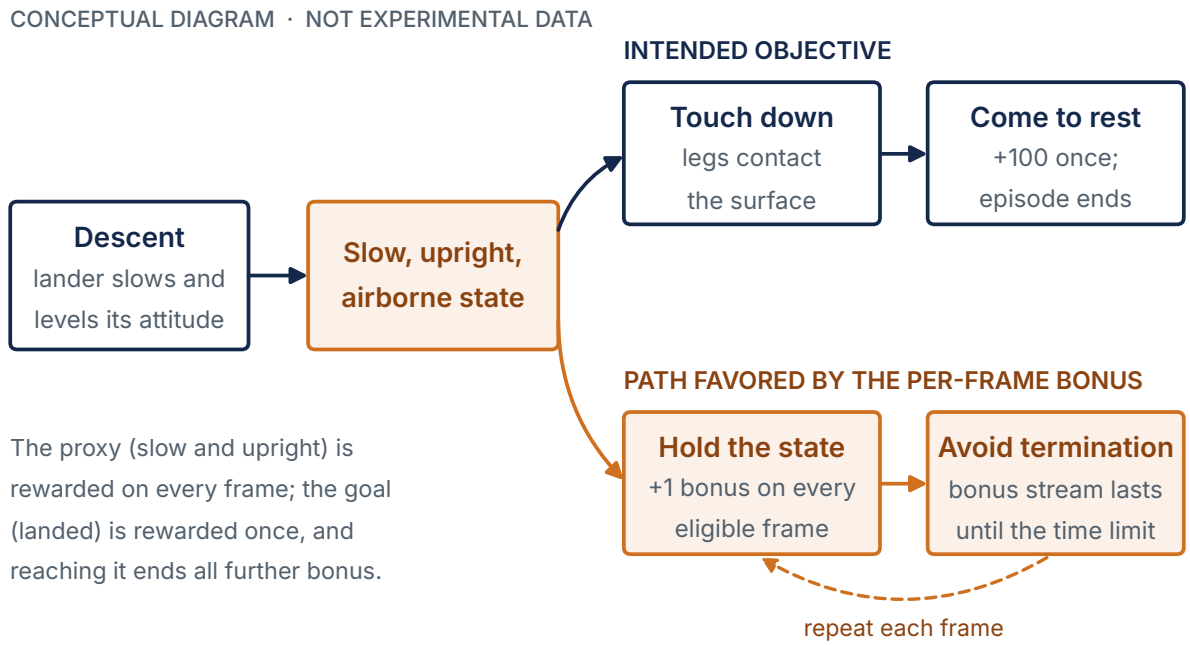


Figure 9. Conceptual mechanism of the hover-reward exploit (conceptual diagram, not experimental data). The per-frame bonus rewards a proxy state repeatedly, while the intended outcome is rewarded once and ends the bonus stream.

is

$$\sum_{k=0}^{n-1} \gamma^k = \frac{1-\gamma^n}{1-\gamma} \longrightarrow \frac{1}{1-\gamma} = 100 \quad \text{as } n \to \infty, \tag{20}$$

which is the same order of magnitude as the $+100$ terminal reward for landing. Three further factors tilt the comparison toward remaining airborne:

- **The time limit is not observable.** The observation (Eq. (9)) contains no clock, and Stable-Baselines3 bootstraps the value of truncated episodes [17, 10]. The learner therefore values a sustainable bonus stream as if it continued past step 1,000, at up to $1/(1-\gamma)$ points.
- **Touchdown carries risk.** Approaching the ground exposes the agent to a crash and the -100 penalty, while holding a slow, upright state in the air does not.
- **The landing reward arrives later.** A landing earns its $+100$ only after the descent, so it is discounted further than bonus frames that begin immediately.

Against this, holding altitude requires main-engine thrust, which costs 0.30 per firing frame in the base reward. Because only one engine can fire per frame, the engine cost never exceeds 0.30 per frame, so a frame spent in the eligible state is worth at least $+0.70$ in training reward, and a sustained stream at least 70 points in discounted value. The engine duty cycle was not recorded, so the exact figure cannot be computed. The analysis shows that the reward makes prolonged slow flight competitive with, and potentially preferable to, landing; it does not prove that PPO converged to an optimal policy for that reward.

12.2 Observed behavior (verified)

The evaluation data for Agent 3 show the following:

- no episode ended at rest (0/100; 95% CI 0% to 3.7%);

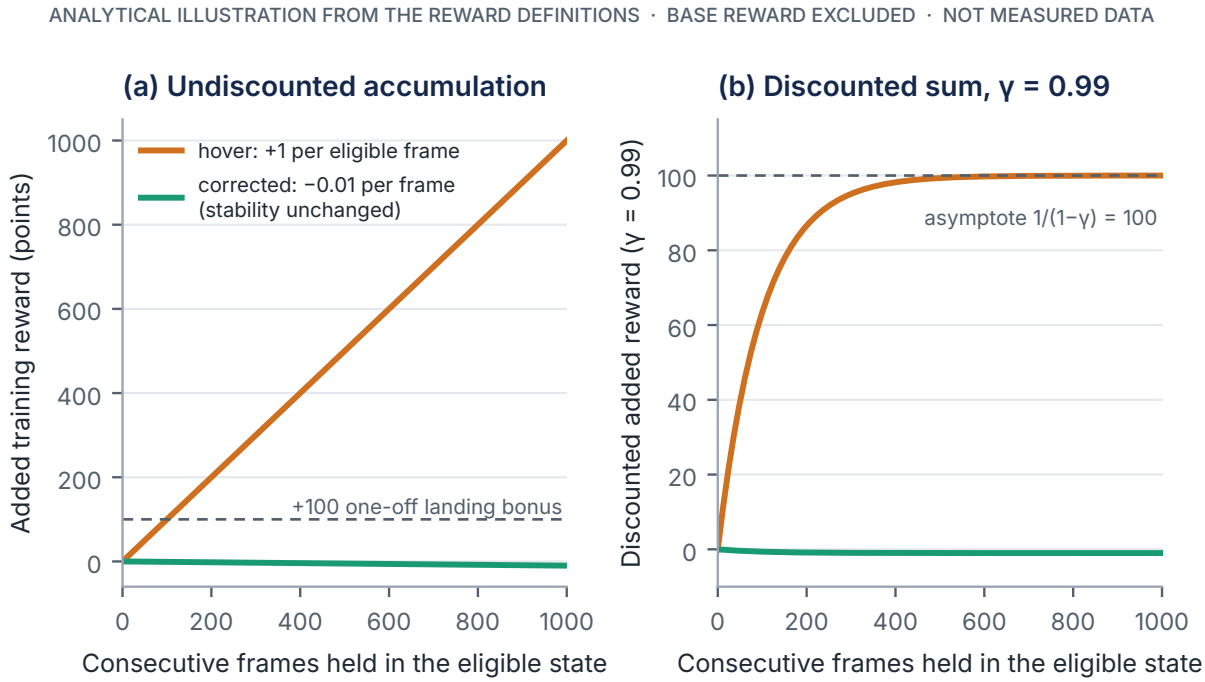


Figure 10. Added training reward from holding the slow, upright state for n consecutive frames under the hover reward and the corrected reward (analytical illustration from Eqs. (13), (16) and (20); base reward excluded; not measured data). (a) Undiscounted sum. (b) Discounted sum with $\gamma = 0.99$.

- 87 of 100 episodes ran to the 1,000-step time limit; in those episodes the lander was in the slow/upright airborne state for a mean of 924 frames;
- across all episodes, a mean of 842.4 frames (86.6%) was spent in that state, compared with 28.1 frames (8.9%) for the baseline;
- time-limit episodes scored −52.2 points on average under the original reward. This is consistent with Eq. (11): an agent that neither lands nor crashes collects no terminal reward and pays engine costs on every firing frame.

12.3 Interpretation and its limits

The combination of (i) an incentive, derived above, to remain in the rewarded state, (ii) complete absence of landings, (iii) episodes that run to the time limit, and (iv) persistent occupancy of exactly the rewarded state, is the signature expected of reward hacking. Keeping speed and tilt below the thresholds for hundreds of consecutive frames requires active control, so the behavior is unlikely to be the result of a policy that simply failed to learn. On this evidence, the hover agent's failure is best interpreted as a learned response to the flawed incentive, matching the definitions of specification gaming in [15, 16].

Three qualifications apply. First, the slow/upright indicator does not establish stationary hovering: the dataset holds no positions, so a lander drifting slowly sideways or descending very slowly would be counted in the same way. Second, no training logs exist to confirm that the agent's training return, including the bonus, rose while its true return fell, which is the formal criterion of Skalse et al. [16]. Third, there is one training run. A second run might land occasionally or hover in a different way. The verified finding is the behavior of this checkpoint; the causal attribution to the reward is a strongly supported interpretation, not a demonstrated effect.

13 Corrected Reward Design and Performance Recovery

13.1 Recovery relative to the hover reward

Replacing the hover bonus with the change-based term of Eq. (16) was followed by a large recovery. On the same seeds, the corrected agent scored 150.8 points more on average (95% CI 126.6 to 174.4) and landed in 55 more episodes. Because the hover agent landed on no seed, all 55 discordant seeds favor the corrected agent (McNemar $p < 0.001$). Its slow/upright airborne time fell by 580.6 frames per episode (95% CI 509.4 to 647.1), and time-limit episodes fell from 87 to 9.

13.2 Comparison with the baseline

The corrected agent did not reach the baseline's performance. Relative to the baseline on the same seeds, it scored 87.3 points less (95% CI 61.9 to 113.9) and landed 20 percentage points less often (95% CI 9 to 31; Holm-adjusted McNemar $p = 0.001$). Figure 11 shows the seed-level picture: both agents landed on 47 seeds, only the baseline on 28, only the corrected agent on 8, and neither on 17.

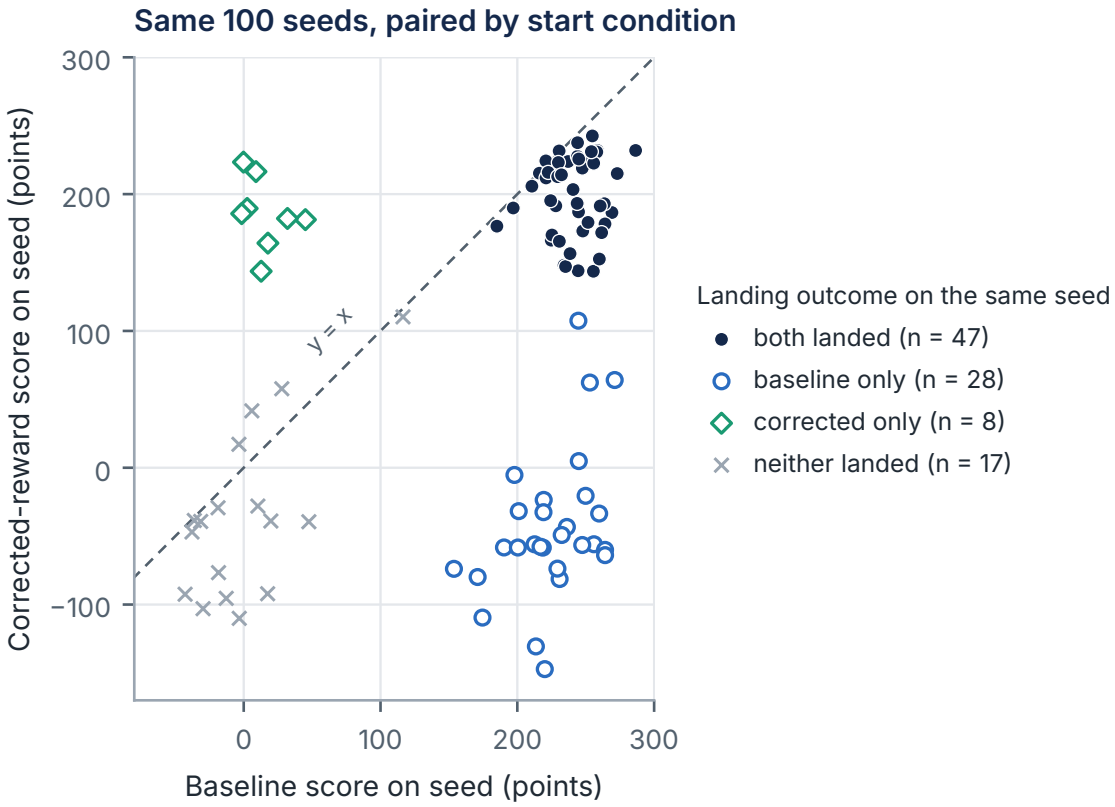


Figure 11. Baseline and corrected-reward scores on each of the 100 shared seeds, marked by joint landing outcome (measured data). Points below the dashed line are seeds on which the baseline scored higher.

The corrected agent's successful landings were also slower and less efficient. They took 513 steps on average, compared with 315 for the baseline, and spent about half of their frames in the slow/upright airborne state. The lower score of its successful landings (197.2 points, compared with 235.0) is consistent with longer flights and therefore more engine use; engine use was not recorded, so this explanation remains an inference.

13.3 Residual behavior

The correction did not eliminate prolonged flight. Nine episodes reached the time limit, and in 13 episodes more than 75% of frames were slow, upright, and airborne. Some of the time-limit episodes scored above 100 points under the original reward, which suggests that the lander reached a favorable state near the ground without coming to rest. The baseline's single time-limit episode (116.4 points) shows a similar pattern. Without trajectory data, the configuration of the lander in these episodes cannot be determined.

13.4 Why the correction fell short of the baseline

Several explanations are compatible with the data, and the experiment cannot distinguish among them:

1. **Training variability.** With one run per condition, the gap of 20 percentage points may partly or wholly reflect the particular runs rather than the reward.
2. **A preference for caution.** By Eq. (17), an episode that ends with large velocity or tilt receives an additional penalty of 2 $S(s_T)$, and Eq. (18) adds a small per-step penalty for instability. Both terms favor slow, careful flight, which matches the longer landings observed.
3. **A changed learning problem.** Additional reward terms change the scale and variance of the returns that the value function must learn. Within a fixed budget of about 500,000 steps, this may slow learning.

> **Assessment of the correction**
>
> The verified result is that the corrected agent landed far more often than the hover agent and less often than the baseline. The reward analysis shows that the correction removes the specific incentive to accumulate bonuses by holding a state. It does not show that the corrected reward is optimal, that it preserves the optimal policy of the original task, or that reward hacking has been eliminated in general.

14 Engineering Discussion and Interpretation

14.1 Strength of the evidence

Table 11 classifies the principal claims of this report by the strength of the evidence behind them. The distinction matters for how the results can be used: verified observations can be cited directly, whereas interpretations and hypotheses should be presented with their qualifications.

14.2 The time penalty

The time-penalty agent landed in 27 episodes compared with the baseline's 75, and its episodes were longer rather than shorter. The reward analysis of Section 8 offers no mechanism by which a penalty of -0.01 per step, worth at most about one point of discounted return, should cause a loss of this size: it is small beside the engine costs and the ± 100 terminal rewards. Its successful landings were in fact slower than the baseline's (533 against 315 steps), the opposite of what the penalty was meant to encourage. The most economical explanation is that the difference arises mainly from training variability between two single runs, and that the penalty, at this magnitude, was too weak to shape behavior noticeably. That explanation is itself a hypothesis. The result does

Table 11. Classification of the principal claims by strength of evidence.

Claim	Status	Basis or requirement
Baseline landed 75/100, time penalty 27/100, hover 0/100, corrected 55/100	Verified	Recomputed from the dataset
Hover agent reached the time limit in 87/100 episodes and spent 86.6% of frames slow/upright/airborne	Verified	Recomputed from the dataset
Corrected agent outperformed hover and underperformed baseline on the same seeds	Verified	Paired tests, Table 10
Hover bonus creates an incentive to prolong episodes; corrected term removes it	Analytical	Eqs. (13)–(20)
Hover agent's failure was caused by the bonus (reward hacking)	Strong interpretation	Converging evidence; needs repeated training and trajectory logs
Time penalty degraded landing performance	Not established	Needs multiple training seeds per condition
Corrected reward causes slower, more cautious landings	Hypothesis	Needs trajectory and engine-use logs
Findings generalize to other coefficients, budgets, or algorithms	Not tested	Needs ablation studies

not show that time penalties reduce performance in general; a penalty large enough to compete with engine costs could plausibly shorten episodes, and that is a question for a coefficient sweep with repeated training.

14.3 Lessons for reward specification

The study illustrates several principles relevant to reward design in robotics and autonomous systems.

- **Reward outcomes and progress, not the occupation of states.** A bonus paid for every step spent in a desirable-looking state can be collected indefinitely, which turns a proxy into a goal. Rewarding changes in a measure, as Agent 4 does, bounds the total bonus by the net improvement, Eq. (17).
- **Prefer potential-based shaping where a guarantee is wanted.** The form of Eq. (4) preserves the optimal policy [12]. Agent 4 departs from it only by a discount residual and a time cost, but those departures are enough to remove the formal guarantee.
- **Evaluate on the true objective.** Scoring all agents with the unmodified reward exposed the hover agent's failure immediately. Had it been evaluated on its own training reward, the same behavior could have appeared successful.
- **Monitor behavior the reward does not score.** The slow/upright indicator and the outcome classes revealed how each agent failed. Metrics of this kind detect specification failures that a single score cannot.
- **Account for episode boundaries.** Time limits, termination rules, and value bootstrapping determine what an agent can gain by delaying the end of an episode. These details are part of the specification.

These principles extend beyond the benchmark. A drone rewarded for stable attitude during a landing approach, or a manipulator rewarded for holding an object above a target, faces a structurally similar incentive: a state that resembles progress but is not completion. In a safety-critical system, a controller that hovers until its energy is exhausted is a failure, regardless of how stable it

appeared during flight. The value of a small, controlled experiment like this one is that the failure can be produced deliberately, observed, and analyzed before it is encountered in a system where it matters.

15 Experimental Limitations and Reproducibility

15.1 Limitations

The limitations below define the scope of the conclusions. They do not invalidate the findings, but each bounds what can be inferred from them (Table 12).

Table 12. Experimental limitations, their effect on interpretation, and possible mitigation.

Limitation	Effect on interpretation	Mitigation
One training run per reward	Reward effect is confounded with run-to-run variance; no interval over training outcomes	5–10 seeds per condition [19]
Training seeds not fixed	Runs cannot be reproduced exactly	Record and fix seeds
Fixed evaluation seeds (100)	Intervals are conditional on these starts; adequate for the large effects, wide for the small ones	More episodes; report per-seed data
Deterministic evaluation	Evaluated controller differs from the stochastic training policy	Also evaluate stochastic actions
500k steps, default hyperparameters	Some agents may be under-trained; the tuned RL Zoo configuration uses 10^6 steps and 16 environments [11]	Learning curves; longer budgets
Single reward coefficients	Sensitivity to the values -0.01, $+1$, and 2 is unknown	Coefficient sweeps
Training vs. evaluation reward	Training-reward performance was not recorded, so proxy–true divergence cannot be shown directly	Log both rewards during training
Landing flag	"Came to rest" is not "landed on the pad"; crash and out-of-bounds are merged	Log terminal cause and position
Slow/upright indicator	Not proof of stationary hovering	Log position, altitude, and actions
Missing artifacts	Training scripts, wrappers, checkpoints, logs, and versions were not available	Publish them with the report

15.2 Reproducibility status

The analysis in this report is fully reproducible: every number, table, and data figure is regenerated from `final_results.csv` by the scripts in the `Analysis` folder (Appendix D), with fixed random seeds for all bootstrap procedures. The training and evaluation of the agents are not reproducible from the supplied material, because the source code, reward wrappers, model checkpoints, and package versions were not included. The evaluation protocol is specified precisely enough (seeds 1000–1099, deterministic actions, unmodified reward) to be re-run once the checkpoints are restored.

16 Recommendations for Future Work

The following extensions are listed in order of the value they add to the present conclusions.

1. **Restore and publish the original artifacts.** Add the training scripts, reward wrappers, checkpoints, and an exact package list (`pip freeze`) to the repository. This step is a prerequisite for independent verification of the training.
2. **Repeat training across seeds.** Train each reward condition with at least five, preferably ten, recorded random seeds, holding all other settings fixed. Report the interquartile mean and stratified bootstrap intervals across runs, following Agarwal et al. [19]. This is the only way to establish whether the reward, rather than the run, caused each difference, and it is essential for the time-penalty result.
3. **Log training curves.** Record both the training (shaped) return and the original-reward return during training. Divergence between the two over time would provide direct evidence of reward hacking in the sense of Skalse et al. [16].
4. **Log trajectories and terminal causes.** Record position, velocity, attitude, leg contact, and action at every step, and the reason each episode ended. Short videos of representative episodes would make the behavioral diagnosis directly visible.
5. **Run ablations.** Sweep the time-penalty magnitude (for example -0.01, -0.1, -0.3), the corrected-reward coefficient (for example 0.5, 1, 2, 5), and compare the heuristic correction with the exactly potential-based variant $2[\gamma S(s_{t+1}) - S(s_t)]$.
6. **Broaden evaluation.** Evaluate stochastic as well as deterministic actions, use more evaluation episodes, and test robustness to wind, turbulence, and altered gravity.

17 Conclusions

Four PPO agents, identical except for their training reward, were evaluated on the same 100 starting conditions under the unmodified LunarLander-v3 reward. The principal conclusions, each stated at the strength the evidence supports, are as follows.

1. **RQ1, baseline.** The baseline agent came to rest in 75 of 100 episodes (95% CI 65.7% to 82.5%) with a mean score of 177.5 points and a median of 227.7 points. It was the best-performing agent on every task metric.
2. **RQ2, time penalty.** The agent trained with a -0.01 per-step penalty landed in 27 episodes and took longer, not shorter, than the baseline. Because the penalty is very small and each reward was trained once, this difference cannot be attributed to the penalty; it does not show that time penalties are harmful in general.
3. **RQ3, reward hacking.** The agent trained with a $+1$ per-step bonus for a slow, upright state never landed, reached the time limit in 87 episodes, and spent 86.6% of its time in the rewarded state. Analysis of the reward shows why: the bonus can be collected repeatedly, while landing pays once and ends the bonus stream. The evidence strongly supports reward hacking as the explanation, while falling short of a causal demonstration.
4. **RQ4, correction.** Rewarding the change in stability rather than its continued satisfaction removes the incentive to accumulate bonuses, because the added reward telescopes to the net change over the episode. The corrected agent landed in 55 episodes, 55 percentage points more than the hover agent on the same seeds, but 20 points fewer than the baseline. The correction mitigated the failure; it did not restore baseline performance or eliminate prolonged flight.

The broader engineering conclusion is that a reward function is a specification that the optimizer will satisfy literally. A term that pays for occupying a state that resembles progress can displace the objective it was meant to support. Reward terms should therefore be analyzed for the behavior they make optimal, and learned controllers should be evaluated against the true objective using behavioral measures that the training reward does not optimize directly.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learning Representations* (*ICLR*), 2016. arXiv:1506.02438.

[4] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," in *Advances in Neural Information Processing Systems*, 2025. arXiv:2407.17032.

[5] Farama Foundation, "Lunar Lander," *Gymnasium Documentation*. [Online]. Available: https://gymnasium.farama.org/environments/box2d/lunar_lander/ (accessed Oct. 21, 2026).

[6] Farama Foundation, *Gymnasium* source code, files gymnasium/envs/box2d/lunar_lander.py and gymnasium/envs/__init__.py. [Online]. Available: https://github.com/Farama-Foundation/Gymnasium (accessed Oct. 21, 2026).

[7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[8] Stable-Baselines3 Developers, "PPO," *Stable-Baselines3 Documentation*. [Online]. Available: https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html (accessed Oct. 21, 2026).

[9] Stable-Baselines3 Developers, "Policy networks," *Stable-Baselines3 Documentation*. [Online]. Available: https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html (accessed Oct. 21, 2026).

[10] Stable-Baselines3 Developers, *Stable-Baselines3* source code, file stable_baselines3/common/on_policy_algorithm.py (timeout bootstrapping in `collect_rollouts`). [Online]. Available: https://github.com/DLR-RM/stable-baselines3 (accessed Oct. 21, 2026).

[11] A. Raffin, *RL Baselines3 Zoo*, hyperparameter file `hyperparams/ppo.yml`, entry `LunarLander-v3`. [Online]. Available: https://github.com/DLR-RM/rl-baselines3-zoo (accessed Oct. 21, 2026).

[12] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. 16th Int. Conf. Machine Learning* (*ICML*), 1999, pp. 278–287.

[13] J. Randløv and P. Alstrøm, "Learning to drive a bicycle using reinforcement learning and shaping," in *Proc. 15th Int. Conf. Machine Learning* (*ICML*), 1998, pp. 463–471.

[14] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," arXiv:1606.06565, 2016.

[15] V. Krakovna *et al.*, "Specification gaming: The flip side of AI ingenuity," DeepMind Blog, Apr. 21, 2020. [Online]. Available: https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/

[16] J. Skalse, N. Howe, D. Krasheninnikov, and D. Krueger, "Defining and characterizing reward gaming," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 9460–9471. arXiv:2209.13085 (preprint title: "Defining and characterizing reward hacking").

[17] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time limits in reinforcement learning," in *Proc. 35th Int. Conf. Machine Learning* (*ICML*), PMLR vol. 80, 2018, pp. 4045–4054.

[18] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, 2018, doi: 10.1609/aaai.v32i1.11694.

[19] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 29304–29320.

[20] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[21] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[22] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[23] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[24] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

Technical Appendices

Appendix A. Dataset Description and Integrity Checks

The primary record, `final_results.csv`, contains one row per evaluation episode (Table 13). Table 14 lists the automated checks applied before any analysis; all passed.

Table 13. Fields of `final_results.csv`.

Field	Type	Definition
`model`	text	Agent label: Original PPO, Time Penalty, Hover Reward, Corrected Reward
`episode`	integer	Episode index, 1–100
`seed`	integer	Environment reset seed, 999 + episode
`score`	real	Undiscounted return under the unmodified reward (points)
`duration_frames`	integer	Steps until termination or truncation
`successful_landing`	0/1	Terminated with final base reward $\geq +100$ (came to rest)
`slow_upright_airborne_frames`	integer	Non-terminal steps with $\lvert v_x\rvert, \lvert v_y\rvert < 0.2$, $\lvert\theta\rvert < 0.15$, both legs off the ground

Table 14. Integrity checks on the evaluation dataset.

Check performed on final_results.csv	Result
Exact column set	PASS
400 rows	PASS
No missing values	PASS
Four expected model labels	PASS
100 episodes per model	PASS
Seeds 1000-1099 for every model	PASS
Seed = 999 + episode	PASS
Success flag is binary	PASS
Duration within 1..1000	PASS
Slow/upright frames $\leq$ duration	PASS
No success at the 1000-frame limit	PASS
Every success scores above 100	PASS

Appendix B. Derivations

B.1 Telescoping of the corrected term. Let $F_{t+1} = 2[S(s_{t+1}) - S(s_t)] - 0.01$. For an episode of T steps,

$$\sum_{t=0}^{T-1} F_{t+1} = 2\sum_{t=0}^{T-1}\big[S(s_{t+1}) - S(s_t)\big] - 0.01\,T = 2\big[S(s_T) - S(s_0)\big] - 0.01\,T,$$

since every intermediate $S(s_t)$ appears once with each sign. This is Eq. (17).

B.2 Relation to potential-based shaping. With $\Phi = 2S$, the potential-based term is $\gamma\Phi(s') - \Phi(s) = 2\gamma S(s') - 2S(s)$. Subtracting it from $2S(s') - 2S(s)$ leaves $2(1-\gamma)S(s')$, which gives Eq. (18). For $\gamma = 0.99$ the residual is $0.02\,S(s') \in [-0.02\,(|v_x| + |v_y| + |\theta|),\, 0]$.

B.3 Discounted value of a constant bonus. A bonus b received for n consecutive steps has discounted value $b\sum_{k=0}^{n-1}\gamma^k = b(1-\gamma^n)/(1-\gamma)$. For $b = 1$ and $\gamma = 0.99$ this is 63.4 at $n = 100$, 99.996 at $n = 1{,}000$, and tends to 100, Eq. (20). For the time penalty, $b = -0.01$, the limit is -1.

B.4 Base return of a non-terminating episode. For an episode truncated at $T = 1{,}000$ steps, no terminal reward is paid and Eq. (11) telescopes to $R = \Psi(s_T) - \Psi(s_0) - 0.30\,M - 0.03\,N$, where M and N are the numbers of main- and side-engine frames. A lander that ends near the pad, slow and upright, recovers the initial deficit in Ψ but pays for every engine frame, which accounts for the negative scores of the hover agent's time-limit episodes.

Appendix C. Observation Scaling and Threshold Conversion

The environment uses FPS $= 50$, SCALE $= 30$, and a viewport of 600×400 pixels [6]. Observed velocities are $v_x = \dot{x}\,(W/2\,\text{SCALE})/\text{FPS} = \dot{x}\cdot 10/50 = 0.2\,\dot{x}$ and $v_y = \dot{y}\,(H/2\,\text{SCALE})/\text{FPS} = \dot{y}\cdot 6.67/50 \approx 0.133\,\dot{y}$, where $\dot{x}, \dot{y}$ are Box2D velocities in simulation units per second. The threshold $|v_x| < 0.2$ therefore corresponds to $|\dot{x}| < 1.0$ unit/s, and $|v_y| < 0.2$ to $|\dot{y}| < 1.5$ units/s. The angle is not rescaled: $|\theta| < 0.15$ rad (8.6°). The stability measure of Eq. (15) adds these normalized quantities, so its three terms have different physical weights.

Appendix D. Reproduction of the Analysis

The analysis requires Python 3.10 or later with `pandas`, `numpy`, `scipy`, and `matplotlib`. From the `Analysis` folder of the portfolio, three commands are run in order:

1. `python verify_and_analyze.py`
2. `python make_figures.py`
3. `python make_tables.py`

These run the integrity checks, write all statistics to `Analysis/outputs/`, regenerate every data figure in `Figures/`, and write the numeric tables used in this report. Bootstrap procedures use 10,000 resamples and the fixed generator seed 20261021; re-running reproduces every interval exactly. Original materials are kept unchanged in `Original_Supplied_Materials/`, separate from the newly written analysis code.

Appendix E. Cross-Check of the Preliminary Summary

Table 15 compares the preliminary summary supplied with the project against the recomputed values. All twelve values agree. The supplied `summary_statistics.csv` also agrees with the recomputation for every reported column; the bootstrap intervals in the supplied `paired_comparisons.csv` differ from those in Table 10 by less than one point, as expected from a different number of resamples.

Table 15. Preliminary summary values versus values recomputed from the raw dataset.

Agent	Brief score	Recomputed	Brief landing	Recomputed	Brief frames	Recomputed	Status
1 Baseline	177.53	177.530	75%	75/100	28.14	28.14	Match
2 Time penalty	−14.98	−14.983	27%	27/100	148.82	148.82	Match
3 Hover reward	−60.63	−60.634	0%	0/100	842.38	842.38	Match
4 Corrected reward	90.21	90.213	55%	55/100	261.74	261.74	Match